

CS 3502 Project 1

Database Records

Nicholas Carrasquilla

July 16, 2026

1 Introduction

This program demonstrates basic multithreading concepts using a simple database record system. The project is divided into four phases. Each phase builds on the previous one by first demonstrating race conditions, then protecting shared resources with mutexes, creating a deadlock, and finally resolving the deadlock using ordered locking.

2 Phase 1

The first phase creates multiple worker threads that update shared database records without any synchronization. Since several threads can modify the same record at the same time, race conditions occur and the final values are often incorrect. Running the program multiple times produces different results because thread scheduling is unpredictable.

```
Jul 16 01:25
nicholation@Ubuntu:~/cs3502-project1
~/cs3502-project1

nicholation@Ubuntu:~/cs3502-project1$ ./phase1
=====
Phase 1: Unsynchronized Database Updates
=====

Workers: 4
Updates per worker: 10000
Database records: 2

Worker 2 started with pthread ID 138136659158720
Worker 1 started with pthread ID 138136667551424
Worker 4 started with pthread ID 138136642373312
Worker 3 started with pthread ID 138136650766016
Worker 4: Record 1 changed from 0 to 1
Worker 4: Record 2 changed from 0 to 1
Worker 4: Record 1 changed from 1 to 2
Worker 1: Record 1 changed from 0 to 1
Worker 1: Record 2 changed from 12 to 13
Worker 1: Record 1 changed from 1 to 2
Worker 2: Record 1 changed from 0 to 1
Worker 2: Record 2 changed from 24 to 25
Worker 2: Record 1 changed from 13 to 14
Worker 3: Record 1 changed from 0 to 1
Worker 3: Record 2 changed from 36 to 37
Worker 3: Record 1 changed from 25 to 26
Worker 3 finished
Worker 1 finished
Worker 2 finished
Worker 4 finished

=====
Phase 1 Results
=====

Record 1
Expected value:      20000
Actual value:       9521
Recorded updates:   19964
Result: RACE CONDITION DETECTED

Record 2
Expected value:      20000
Actual value:       9940
Recorded updates:   19991
Result: RACE CONDITION DETECTED

Expected total updates: 40000
Actual total value:    19461
Lost updates:         20539
Execution time:       0.404335 seconds

Race condition demonstrated successfully.
nicholation@Ubuntu:~/cs3502-project1$
```

Figure 1: Phase 1 program output

3 Phase 2

In the second phase, a mutex is added to each database record. Before a worker updates a record, it locks the mutex and unlocks it after the update is complete. This prevents multiple threads from modifying the same record simultaneously. The final values remain correct every time the program is executed.

```
Jul 16 01:26
nicholation@Ubuntu:~/cs3502-project1
~/cs3502-project1

nicholation@Ubuntu:~/cs3502-project1$ ./phase2
=====
Phase 2: Synchronized Database Updates
=====

Workers: 4
Updates per worker: 10000
Database records: 2

Worker 1 started with pthread ID 125985655617216
Worker 2 started with pthread ID 125985647224512
Worker 4 started with pthread ID 125985630439104
Worker 3 started with pthread ID 125985638031008
Worker 1: Record 1 changed from 0 to 1
Worker 1: Record 2 changed from 0 to 1
Worker 1: Record 1 changed from 1 to 2
Worker 2: Record 1 changed from 13 to 14
Worker 2: Record 2 changed from 25 to 26
Worker 3: Record 1 changed from 39 to 40
Worker 3: Record 2 changed from 39 to 40
Worker 2: Record 1 changed from 40 to 41
Worker 4: Record 1 changed from 52 to 53
Worker 4: Record 2 changed from 52 to 53
Worker 4: Record 1 changed from 53 to 54
Worker 3: Record 1 changed from 54 to 55
Worker 3 finished
Worker 1 finished
Worker 2 finished
Worker 4 finished

=====
Phase 2 Results
=====

Record 1
Expected value: 20000
Actual value: 20000
Recorded updates: 20000
Result: CORRECT

Record 2
Expected value: 20000
Actual value: 20000
Recorded updates: 20000
Result: CORRECT

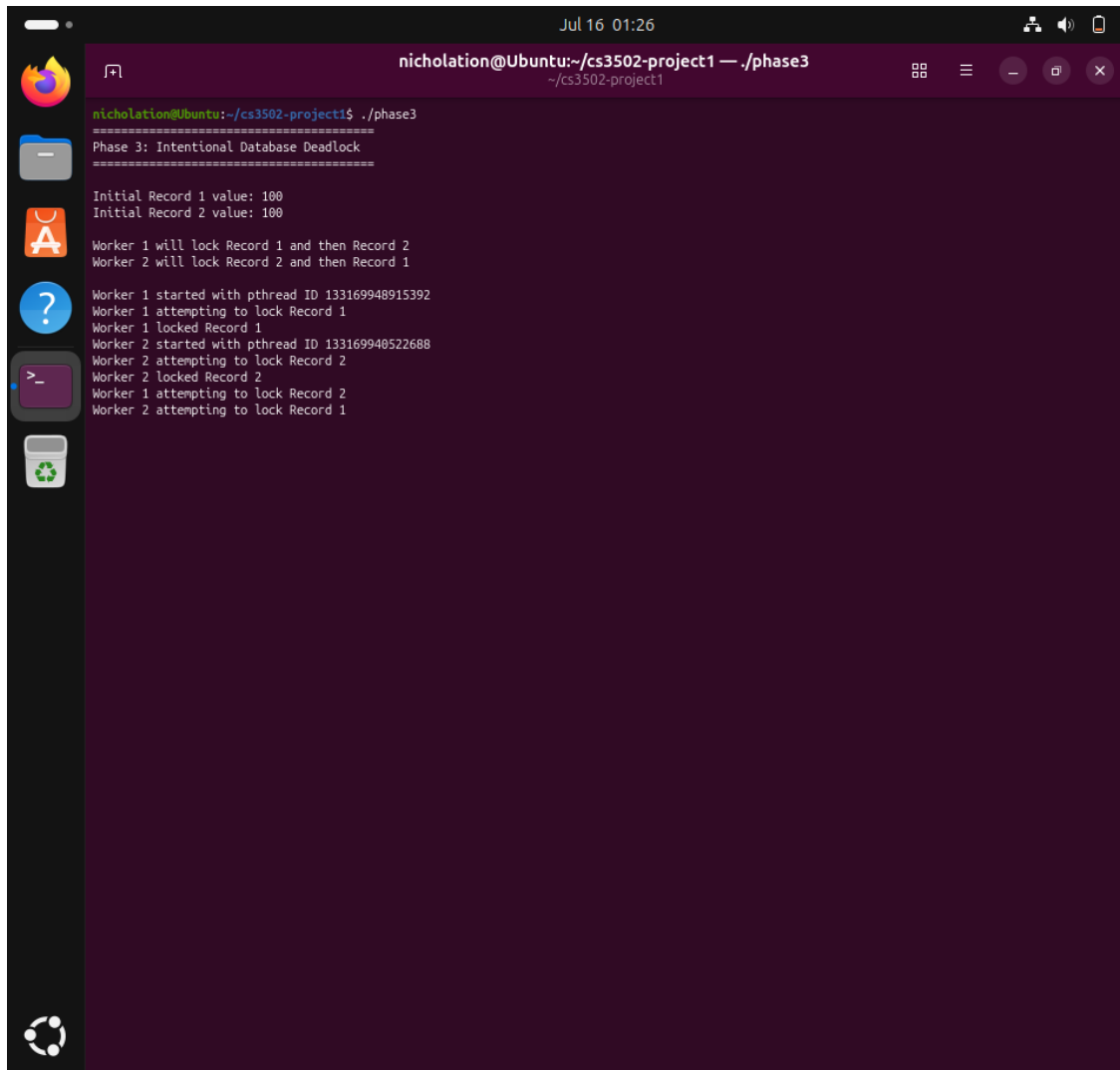
Expected total updates: 40000
Actual total value: 40000
Lost updates: 0
Execution time: 2.774288 seconds

Mutex synchronization worked successfully.
nicholation@Ubuntu:~/cs3502-project1$
```

Figure 2: Phase 2 program output

4 Phase 3

The third phase demonstrates a deadlock. Two worker threads attempt to lock two different records in opposite order. Each thread successfully locks one record but waits forever for the second record, causing the program to stop making progress.



```
nicholation@Ubuntu:~/cs3502-project1$ ./phase3
Phase 3: Intentional Database Deadlock

Initial Record 1 value: 100
Initial Record 2 value: 100

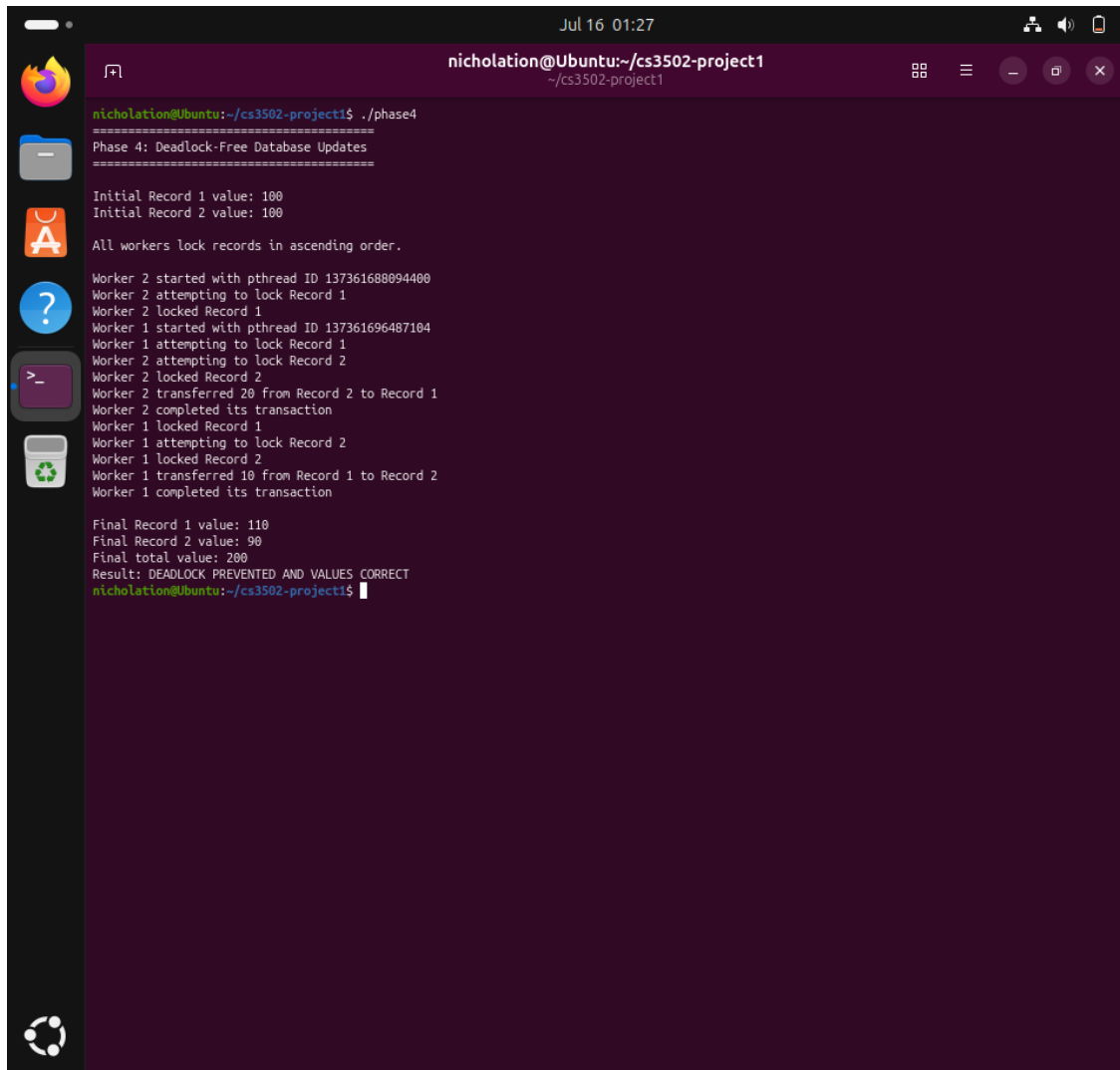
Worker 1 will lock Record 1 and then Record 2
Worker 2 will lock Record 2 and then Record 1

Worker 1 started with pthread ID 133169948915392
Worker 1 attempting to lock Record 1
Worker 1 locked Record 1
Worker 2 started with pthread ID 133169948522688
Worker 2 attempting to lock Record 2
Worker 2 locked Record 2
Worker 1 attempting to lock Record 2
Worker 2 attempting to lock Record 1
```

Figure 3: Phase 3 program output

5 Phase 4

The final phase prevents the deadlock by always locking the records in the same order. Since every thread follows the same locking order, the circular wait condition is removed and all transactions complete successfully.

A terminal window titled 'nicholation@Ubuntu:~/cs3502-project1' with a timestamp 'Jul 16 01:27'. The window shows the output of a program running './phase4'. The output is as follows:

```
nicholation@Ubuntu:~/cs3502-project1$ ./phase4
=====
Phase 4: Deadlock-Free Database Updates
=====

Initial Record 1 value: 100
Initial Record 2 value: 100

All workers lock records in ascending order.

Worker 2 started with pthread ID 137361688094400
Worker 2 attempting to lock Record 1
Worker 2 locked Record 1
Worker 1 started with pthread ID 137361696487104
Worker 1 attempting to lock Record 1
Worker 2 attempting to lock Record 2
Worker 2 locked Record 2
Worker 2 transferred 20 from Record 2 to Record 1
Worker 2 completed its transaction
Worker 1 locked Record 1
Worker 1 attempting to lock Record 2
Worker 1 locked Record 2
Worker 1 transferred 10 from Record 1 to Record 2
Worker 1 completed its transaction

Final Record 1 value: 110
Final Record 2 value: 90
Final total value: 200
Result: DEADLOCK PREVENTED AND VALUES CORRECT
nicholation@Ubuntu:~/cs3502-project1$
```

Figure 4: Phase 4 program output

6 Challenges and Solutions

One challenge was making the race condition occur consistently enough to observe incorrect results. Small delays were added to increase thread interleaving. Another challenge was creating a reliable deadlock. This was solved by having two threads acquire locks in opposite order. The deadlock was then removed by changing both threads to acquire locks using the same order.

7 Performance Observations

The unsynchronized version executes slightly faster because no locking is required, but the results are unreliable. The synchronized version introduces a small amount of overhead due to mutex locking, but it consistently produces the correct results. The deadlock example shows

how incorrect locking strategies can prevent progress, while the final phase demonstrates that ordered locking avoids deadlock without significantly affecting performance.

8 Conclusion

This project demonstrated several important multithreading concepts including race conditions, mutual exclusion, deadlock creation, and deadlock prevention. The final implementation successfully synchronized access to shared data and prevented deadlocks by using a consistent locking strategy.